CS-3510-B Problem Set 1

Elijah Martin McCauley

TOTAL POINTS

51 / 60

QUESTION 1

1 1 **10 / 10**

✓ **- 0 pts** *Correct (d,h,b,a,c,f,g,e,i)*

- 1 pts One ranking incorrect

- 2 pts 2-3 rankings incorrect

- 4 pts 4-5 rankings incorrect

- 7 pts More than half of the permutation is incorrect

- 10 pts Incorrect answer/Missing

QUESTION 2

2 15 pts

2.1 (a.) **5 / 5**

✓ **- 0 pts** *Correct (False)*

- 2 pts Minor mistake

- 3 pts The example shown is incorrect.

- 4 pts No work shown

- 5 pts Missing/proved the statement as true

2.2 (b.) **0 / 5**

- 0 pts Correct (False)

- 2 pts Minor mistake

- 3 pts The example shown is incorrect.

- 4 pts No work shown

✓ **- 5 pts** *Missing/proved the statement as true*

2.3 (c.) **5 / 5**

✓ **- 0 pts** *Correct (False)*

- 2 pts Minor mistake

- 3 pts The example shown is incorrect.

- 4 pts No work shown

- 5 pts Missing/proved the statement as true

Negative runtimes are not possible. Since Big-Oh is supposed to be an approximation of the number of operations an algorithm performs related to its domain size then it would not make sense to describe an algorithm as using a negative number of operations.

We'll accept your response for now, but make sure in the future you pick a function that has a non-negative runtime. For ex. f(n) = n and g(n) = n^(2*sin n).

QUESTION 3

3 10 pts

3.1 (a.) **5 / 5**

✓ **- 0 pts** *Correct*

- 1 pts Correct T(n), but didn't solve to O(n^3)

- 1 pts Minor mistake

- 3 pts Incorrect T(n) or partially incorrect

- 5 pts Incorrect or mostly incorrect/Missing

3.2 (b.) **5 / 5**

✓ **- 0 pts** *Correct*

- **3 pts** Missing/Incorrect explanation
- **5 pts** Incorrect/Missing

QUESTION 4

4 10 pts

4.1 (a.) 4 / 4

✓ **- 0 pts** *Correct (O(n^2))*
- **1 pts** Minor mistake
- **2 pts** Incorrect Recurrence Relation/No work shown
- **2 pts** Correct recurrence, but incorrect runtime
- **4 pts** Incorrect/Missing

4.2 (b.) 3 / 4

✓ **- 0 pts** *Correct (O(n^2))*
- **2 pts** Incorrect Recurrence Relation/No work shown
- **2 pts** Correct recurrence, but incorrect runtime
- **4 pts** Incorrect/Missing

- 1 *Point adjustment*

You should use O() notation!

4.3 (c.) 2 / 2

✓ **- 0 pts** *Correct (Both are same)*
- **2 pts** Incorrect/Missing

QUESTION 5

5 5 12 / 15

- **0 pts** Correct
- **13 pts** Missing or incorrect algorithm
- **10 pts** Algorithm is mostly incorrect/ not using D&C
- **5 pts** Algorithm is not efficient/ partially correct
- **3 pts** Minor mistake in algorithm
- **5 pts** Missing or incorrect Master Theorem proof
- **2 pts** Missing or incorrect time complexity answer

✓ **- 2 pts** *Minor mistake in Master Theorem proof*
- **2 pts** Ambiguous Algorithimic explanation
- **1 pts** Missing Inclusive Case
- **3 pts** Proof of Correctness missing
- **15 pts** No answer

- 1 *Point adjustment*

log 2 (2) = 1 not 0. so wrong recurrence relation and wrong usage of masters theorem. as logb (a) = 1 > d=0 then runtime = O(n). No mention of thw base cases in the algorithm. -1

Name: Elijah McCauley

1.) (10 points) For the following list of functions, cluster the functions of the same order (i.e., f and g are in the same group if and only if $f = \Theta(g)$ into one group, and then rank the groups in increasing order. You do not have to justify your answer.

(a.) $n\sqrt{n}$

(b.) $n^{1.0001}$

(c.) $n^3 + 3n^2 + n$

(d.) 2^{100}

(e.) $n^{\log n}$

(f.) $1024^{\log n}$

(g.) $(\log n)^{\log n}$

(h.) $16^{\log \log n}$

(i.) $n \log n + 2023n!$

Solution: d, h, b, a, c, f, g, e, i

1 1 **10 / 10**

✓ **- 0 pts** *Correct (d,h,b,a,c,f,g,e,i)*

- 1 pts One ranking incorrect

- 2 pts 2-3 rankings incorrect

- 4 pts 4-5 rankings incorrect

- 7 pts More than half of the permutation is incorrect

- 10 pts Incorrect answer/Missing

2.) (15 points) Prove or disprove (with a counterexample) the following claims:

(a.) If $f(n) = \mathcal{O}(p(n))$ and $g(n) = \mathcal{O}(q(n))$, then $f(n) - g(n) = \mathcal{O}(p(n) - q(n))$

Solution: Counter Example: $f(n) = n^2 + n = \mathcal{O}(n^2)$, so $p(n) = n^2$ and $g(n) = n^2 - n = \mathcal{O}(n^2)$, so $q(n) = n^2$, then $f(n) - g(n) = 2n$, $\mathcal{O}(p(n) - q(n)) = \mathcal{O}(0)$ and $2n \neq \mathcal{O}(0)$

(b.) If $f(n) = \mathcal{O}(g(n))$, then $2^{f(n)} = \mathcal{O}(2^{g(n)})$

Solution: Suppose we have a function $f(n)$. To find the Big-O of the function, we must find positive constants c and k such that $0 \leq f(n) \leq cg(n)$ for all $n \geq k$. Once we find c and k that fulfill this, then we can say that $f(n) = \mathcal{O}(g(n))$. If we then put both sides of the inequality as exponents of 2, the inequality becomes: $2^{f(n)} \leq 2^{cg(x)}$. Because this inequality is also true for some positive constants c and k such that $0 \leq 2^{f(n)} \leq c2^{g(n)}$ for all $n \geq k$. We can say that $2^{f(n)} = \mathcal{O}(2^{g(n)})$.

(c.) There are no functions $f(n)$ and $g(n)$ such that $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$.

Solution: If $f(n) = log(n)$, and $g(n) = nsin(n)$ then $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$. Because $nsin(n)$ has an upper bound greater than $log(n)$ but also is non-monotonic meaning it fluctuates between being greater than, and less then $log(n)$, there is no constant k after which $cg(n)$ will always be $\geq f(n)$ and vice versa there is no constant k after which $cf(n)$ will always be $\geq g(n)$.

2.1 (a.) **5 / 5**

✓ **- 0 pts** *Correct (False)*

- 2 pts Minor mistake

- 3 pts The example shown is incorrect.

- 4 pts No work shown

- 5 pts Missing/proved the statement as true

2.) (15 points) Prove or disprove (with a counterexample) the following claims:

(a.) If $f(n) = \mathcal{O}(p(n))$ and $g(n) = \mathcal{O}(q(n))$, then $f(n) - g(n) = \mathcal{O}(p(n) - q(n))$

Solution: Counter Example: $f(n) = n^2 + n = \mathcal{O}(n^2)$, so $p(n) = n^2$ and $g(n) = n^2 - n = \mathcal{O}(n^2)$, so $q(n) = n^2$, then $f(n) - g(n) = 2n$, $\mathcal{O}(p(n) - q(n)) = \mathcal{O}(0)$ and $2n \neq \mathcal{O}(0)$

(b.) If $f(n) = \mathcal{O}(g(n))$, then $2^{f(n)} = \mathcal{O}(2^{g(n)})$

Solution: Suppose we have a function $f(n)$. To find the Big-O of the function, we must find positive constants c and k such that $0 \leq f(n) \leq cg(n)$ for all $n \geq k$. Once we find c and k that fulfill this, then we can say that $f(n) = \mathcal{O}(g(n))$. If we then put both sides of the inequality as exponents of 2, the inequality becomes: $2^{f(n)} \leq 2^{cg(x)}$. Because this inequality is also true for some positive constants c and k such that $0 \leq 2^{f(n)} \leq c2^{g(n)}$ for all $n \geq k$. We can say that $2^{f(n)} = \mathcal{O}(2^{g(n)})$.

(c.) There are no functions $f(n)$ and $g(n)$ such that $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$.

Solution: If $f(n) = log(n)$, and $g(n) = nsin(n)$ then $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$. Because $nsin(n)$ has an upper bound greater than $log(n)$ but also is non-monotonic meaning it fluctuates between being greater than, and less then $log(n)$, there is no constant k after which $cg(n)$ will always be $\geq f(n)$ and vice versa there is no constant k after which $cf(n)$ will always be $\geq g(n)$.

2.2 (b.) **0 / 5**

- **0 pts** Correct (False)
- **2 pts** Minor mistake
- **3 pts** The example shown is incorrect.
- **4 pts** No work shown

✓ **- 5 pts** *Missing/proved the statement as true*

2.) (15 points) Prove or disprove (with a counterexample) the following claims:

(a.) If $f(n) = \mathcal{O}(p(n))$ and $g(n) = \mathcal{O}(q(n))$, then $f(n) - g(n) = \mathcal{O}(p(n) - q(n))$

Solution: Counter Example: $f(n) = n^2 + n = \mathcal{O}(n^2)$, so $p(n) = n^2$ and $g(n) = n^2 - n = \mathcal{O}(n^2)$, so $q(n) = n^2$, then $f(n) - g(n) = 2n$, $\mathcal{O}(p(n) - q(n)) = \mathcal{O}(0)$ and $2n \neq \mathcal{O}(0)$

(b.) If $f(n) = \mathcal{O}(g(n))$, then $2^{f(n)} = \mathcal{O}(2^{g(n)})$

Solution: Suppose we have a function $f(n)$. To find the Big-O of the function, we must find positive constants c and k such that $0 \leq f(n) \leq cg(n)$ for all $n \geq k$. Once we find c and k that fulfill this, then we can say that $f(n) = \mathcal{O}(g(n))$. If we then put both sides of the inequality as exponents of 2, the inequality becomes: $2^{f(n)} \leq 2^{cg(x)}$. Because this inequality is also true for some positive constants c and k such that $0 \leq 2^{f(n)} \leq c2^{g(n)}$ for all $n \geq k$. We can say that $2^{f(n)} = \mathcal{O}(2^{g(n)})$.

(c.) There are no functions $f(n)$ and $g(n)$ such that $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$.

Solution: If $f(n) = log(n)$, and $g(n) = nsin(n)$ then $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$. Because $nsin(n)$ has an upper bound greater than $log(n)$ but also is non-monotonic meaning it fluctuates between being greater than, and less then $log(n)$, there is no constant k after which $cg(n)$ will always be $\geq f(n)$ and vice versa there is no constant k after which $cf(n)$ will always be $\geq g(n)$.

2.3 (C.) **5 / 5**

✓ **- 0 pts** *Correct (False)*

- 2 pts Minor mistake

- 3 pts The example shown is incorrect.

- 4 pts No work shown

- 5 pts Missing/proved the statement as true

Negative runtimes are not possible. Since Big-Oh is supposed to be an approximation of the number of operations an algorithm performs related to its domain size then it would not make sense to describe an algorithm as using a negative number of operations.
We'll accept your response for now, but make sure in the future you pick a function that has a non-negative runtime. For ex. f(n) = n and g(n) = n^(2*sin n).

3.) (10 points) Assume n is a power of 2. Assume we are given an algorithmic routine $f(n)$ as follows:

```
function f(n):
   if n>1:
      for i in range(n):
         for j in range(n*n):
            print("CS 1332")
      f(n/2)
      f(n/2)
   else:
      print("CS 3510")
```

(a.) What is the running time for this function $f(n)$? Justify your answer. (Hint: Compute $T(n)$ first)

Solution: Starting by solving for $T(n) = aT(n/b) + \mathcal{O}(n^d)$. There are two recursive calls, so a = 2. Each call divides n in half, so b = 2. The $\mathcal{O}$ of the non-recursive part of the function is $\mathcal{O}(n^3)$. So $T(n) = 2T(n/2) + \mathcal{O}(n^3)$. From here, we can use Master's Theorem. d = 3, a = 2, b = 2, so $d = 3 > \log_2(2) = 1$. This means that the run time equals $\mathcal{O}(n^d) = \mathcal{O}(n^3)$.

(b.) How many times will this function print "CS 3510"? Please provide the exact number in terms of the input n. Justify your answer.

Solution: This function will print "CS 3510" n times. This is because the base case of the function is when n = 1, and causes "CS 3510" to print. The number of leaves (which is equivalent to the base case) can be calculated using $n^{\log_b(a)}$ and the same a and b from Master's Theorem, so $n^{\log_b(a)} = n^{\log_2(2)} = n^1$.

3.1 (a.) **5 / 5**

✓ **- 0 pts** *Correct*

- 1 pts Correct T(n), but didn't solve to O(n^3)

- 1 pts Minor mistake

- 3 pts Incorrect T(n) or partially incorrect

- 5 pts Incorrect or mostly incorrect/Missing

3.) (10 points) Assume n is a power of 2. Assume we are given an algorithmic routine $f(n)$ as follows:

```
function f(n):
   if n>1:
      for i in range(n):
         for j in range(n*n):
            print("CS 1332")
      f(n/2)
      f(n/2)
   else:
      print("CS 3510")
```

(a.) What is the running time for this function $f(n)$? Justify your answer. (Hint: Compute $T(n)$ first)

Solution: Starting by solving for $T(n) = aT(n/b) + \mathcal{O}(n^d)$. There are two recursive calls, so a = 2. Each call divides n in half, so b = 2. The $\mathcal{O}$ of the non-recursive part of the function is $\mathcal{O}(n^3)$. So $T(n) = 2T(n/2) + \mathcal{O}(n^3)$. From here, we can use Master's Theorem. d = 3, a = 2, b = 2, so $d = 3 > \log_2(2) = 1$. This means that the run time equals $\mathcal{O}(n^d) = \mathcal{O}(n^3)$.

(b.) How many times will this function print "CS 3510"? Please provide the exact number in terms of the input n. Justify your answer.

Solution: This function will print "CS 3510" n times. This is because the base case of the function is when n = 1, and causes "CS 3510" to print. The number of leaves (which is equivalent to the base case) can be calculated using $n^{\log_b(a)}$ and the same a and b from Master's Theorem, so $n^{\log_b(a)} = n^{\log_2(2)} = n^1$.

3.2 (b.) **5 / 5**

✓ **- 0 pts** *Correct*

- 3 pts Missing/Incorrect explanation

- 5 pts Incorrect/Missing

4.) (10 points) Suppose one of our TAs, Saahir, is working independently to solve a problem using Divide & Conquer. He designed a solution that breaks the problem into three sub-problems, each of size $n/3$, and combines the results in time $n^2 + n \log n + 1$. Another TA, Bharat, was able to solve the same problem using four sub-problems, each of size $n/2$, and combined the results in $\log n + 3n$ to get the final answer.

(a.) Write the recurrence relation and runtime of Saahir's algorithm.

Solution: The recurrence relation is $T(n) = 3T(n/3)$ because the branching factor (a) is 3 and the size of subproblem (b) is also 3. We can then apply Master's Theorem, since we know the combining time is $n^2 + n \log n + 1$. Therefore, $T(n) = 3T(n/3) + \mathcal{O}(n^2)$. We know that d = 2 because the cost to recombine's largest power is n^2. We then compare d to $\log_b(a)$. $d = 2 > \log_2(2) = 1$, so the runtime is simply n^d which is n^2.

(b.) Write the recurrence relation and runtime of Bharat's algorithm.

Solution: The recurrence relation is $T(n) = 4T(n/2)$ because the branching factor (a) is 4 and the size of subproblem (b) is 2. We can then apply Master's Theorem, since we know the combining time is $\log n + 3n$. Therefore, $T(n) = 3T(n/3) + \mathcal{O}(n)$. We know that d = 1 because the cost to recombine's largest power is n^1. We then compare d to $\log_b(a)$. $d = 1 < \log_2(4) = 2$, so the runtime is simply $n^{\log_b(a)}$ which is $n^{\log_2(4)} = n^2$.

(c.) Determine whose algorithm was the fastest.

Solution: The algorithms are the same speed because they both have a total runtime of $\mathcal{O}(n^2)$.

4.1 (a.) **4 / 4**

✓ **- 0 pts** *Correct (O(n^2))*

- 1 pts Minor mistake

- 2 pts Incorrect Recurrence Relation/No work shown

- 2 pts Correct recurrence, but incorrect runtime

- 4 pts Incorrect/Missing

4.) (10 points) Suppose one of our TAs, Saahir, is working independently to solve a problem using Divide & Conquer. He designed a solution that breaks the problem into three sub-problems, each of size $n/3$, and combines the results in time $n^2 + n \log n + 1$. Another TA, Bharat, was able to solve the same problem using four sub-problems, each of size $n/2$, and combined the results in $\log n + 3n$ to get the final answer.

(a.) Write the recurrence relation and runtime of Saahir's algorithm.

Solution: The recurrence relation is $T(n) = 3T(n/3)$ because the branching factor (a) is 3 and the size of subproblem (b) is also 3. We can then apply Master's Theorem, since we know the combining time is $n^2 + n \log n + 1$. Therefore, $T(n) = 3T(n/3) + \mathcal{O}(n^2)$. We know that d = 2 because the cost to recombine's largest power is n^2. We then compare d to $\log_b(a)$. $d = 2 > \log_2(2) = 1$, so the runtime is simply n^d which is n^2.

(b.) Write the recurrence relation and runtime of Bharat's algorithm.

Solution: The recurrence relation is $T(n) = 4T(n/2)$ because the branching factor (a) is 4 and the size of subproblem (b) is 2. We can then apply Master's Theorem, since we know the combining time is $\log n + 3n$. Therefore, $T(n) = 3T(n/3) + \mathcal{O}(n)$. We know that d = 1 because the cost to recombine's largest power is n^1. We then compare d to $\log_b(a)$. $d = 1 < \log_2(4) = 2$, so the runtime is simply $n^{\log_b(a)}$ which is $n^{\log_2(4)} = n^2$.

(c.) Determine whose algorithm was the fastest.

Solution: The algorithms are the same speed because they both have a total runtime of $\mathcal{O}(n^2)$.

4.2 (b.) **3 / 4**

✓ **- 0 pts** *Correct (O(n^2))*

- 2 pts Incorrect Recurrence Relation/No work shown

- 2 pts Correct recurrence, but incorrect runtime

- 4 pts Incorrect/Missing

- 1 *Point adjustment*

You should use O() notation!

4.) (10 points) Suppose one of our TAs, Saahir, is working independently to solve a problem using Divide & Conquer. He designed a solution that breaks the problem into three sub-problems, each of size $n/3$, and combines the results in time $n^2 + n \log n + 1$. Another TA, Bharat, was able to solve the same problem using four sub-problems, each of size $n/2$, and combined the results in $\log n + 3n$ to get the final answer.

(a.) Write the recurrence relation and runtime of Saahir's algorithm.

Solution: The recurrence relation is $T(n) = 3T(n/3)$ because the branching factor (a) is 3 and the size of subproblem (b) is also 3. We can then apply Master's Theorem, since we know the combining time is $n^2 + n \log n + 1$. Therefore, $T(n) = 3T(n/3) + \mathcal{O}(n^2)$. We know that d = 2 because the cost to recombine's largest power is n^2. We then compare d to $\log_b(a)$. $d = 2 > \log_2(2) = 1$, so the runtime is simply n^d which is n^2.

(b.) Write the recurrence relation and runtime of Bharat's algorithm.

Solution: The recurrence relation is $T(n) = 4T(n/2)$ because the branching factor (a) is 4 and the size of subproblem (b) is 2. We can then apply Master's Theorem, since we know the combining time is $\log n + 3n$. Therefore, $T(n) = 3T(n/3) + \mathcal{O}(n)$. We know that d = 1 because the cost to recombine's largest power is n^1. We then compare d to $\log_b(a)$. $d = 1 < \log_2(4) = 2$, so the runtime is simply $n^{\log_b(a)}$ which is $n^{\log_2(4)} = n^2$.

(c.) Determine whose algorithm was the fastest.

Solution: The algorithms are the same speed because they both have a total runtime of $\mathcal{O}(n^2)$.

4.3 (C.) **2 / 2**

✓ **- 0 pts** *Correct (Both are same)*

- 2 pts Incorrect/Missing

5.) (15 points) Suppose you are given the following inputs: a sorted array A, containing n distinct integers, a lower bound l, and an upper bound u, where l and u might not be in the array A.

Design a divide & conquer algorithm in $\mathcal{O}(\log n)$ to find the number of elements in A between l and u, inclusive. Make sure to provide the proof of correctness of your algorithm and the time complexity (using Master's Theorem).

Solution: Basically, we need to run two binary searches. Start by creating two temp index $b = 1$ (the smallest index in the subarray) and $h = arrayLength$. We will start by looking for the smallest number in the array that is less than or equal to l, the lower bound. Assuming arrays starting index is 1, We do this by while $b < h$, we find a mid point, m by adding $(b+h)/2$. If the value at that point is l, return that position. If it is less than l, then b becomes $m + 1$, and the loop repeats. If it is greater than l, then h becomes $m - 1$ and the loop repeats. When the loop finishes, it will either output the index of l, or the index of first element in the array that is greater than l. Repeat this process with u, to output either the index of u or the index of the last element that is less than u. From here, you can simply subtract the index found with l from the index found with u and add 1 that is the number of elements in A between l and u.

Proof of correctness: If the lower bound l and upper bound u are both in the array A, the function will return the positions of l and u in the array, and the function will return the correct count of elements between them.

If l is not in the array, the function will return the position of the first element in the array that is greater l. Similarly, if u is not in the array, the function will return the position of the last element in the array that is less than u. In this case, the function will return the correct count of elements between l and u by subtracting the position of the first element greater than or equal to l from the position of the last element less than or equal to u.

The time complexity can be determined using Master's Theorem through looking at $T(n) = aT(n/b) + \mathcal{O}(n^d)$. The branching factor is 2 because we do the algorithm twice, the size of subarray is 2 because we use $length/2$, and the cost of combination is 0 because we are just saving an index. Comparing d to $\log_2(2)$, you get $0 = 0$, so the runtime is $n^d log(n) = n^0 log(n) = log(n)$.

5 5 **12 / 15**

- **0 pts** Correct
- **13 pts** Missing or incorrect algorithm
- **10 pts** Algorithm is mostly incorrect/ not using D&C
- **5 pts** Algorithm is not efficient/ partially correct
- **3 pts** Minor mistake in algorithm
- **5 pts** Missing or incorrect Master Theorem proof
- **2 pts** Missing or incorrect time complexity answer

✓ **- 2 pts** *Minor mistake in Master Theorem proof*

- **2 pts** Ambiguous Algorithimic explanation
- **1 pts** Missing Inclusive Case
- **3 pts** Proof of Correctness missing
- **15 pts** No answer

- 1 *Point adjustment*

log 2 (2) = 1 not 0. so wrong recurrence relation and wrong usage of masters theorem. as logb (a) = 1 > d=0 then runtime = O(n). No mention of thw base cases in the algorithm. -1

CS 3510: Design & Analysis of Algorithms 01/12/2023

Problem Set 1: Big-$\mathcal{O}$ and Divide & Conquer Warm-up

Professor Abrahim Ladha Due: 01/19/2023 11:59pm

- **Please TYPE your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.**
- **Your solutions must be in plain English and mathematical expressions. Show the running time using the Master Theorem (wherever applicable).**
- **Unless otherwise stated, use *log* to the base 2.**
- **Unless otherwise stated, the fastest (and correct) algorithms will receive more credit. If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.**
- **Do not use pseudocode. Your answer will receive zero credit even if the pseudocode is correct.**